Pipeline de Microscopía con Inteligencia Artificial

Documentación Técnica: Modelos de Mejora y Segmentación
CubeSat EdgeAI Payload

Proyecto CubeSat EdgeAI

Abril 2026

Índice

1. Visión general del pipeline **2**

2. Calibración **2**
2.1. Método automático (imagen de regla) . . . 2
2.2. Método manual . . . 2

3. Preprocesamiento **3**
3.1. CLAHE (Contrast Limited Adaptive Histogram Equalization) . . . 3
3.2. Denoising clásico . . . 3

4. Mejora por IA: Noise2Void (N2V) **3**
4.1. Problema que resuelve . . . 3
4.2. Principio fundamental: self-supervised denoising . . . 3
4.3. Arquitectura: U-Net . . . 4
4.4. Entrenamiento blind-spot . . . 4
4.5. Inferencia . . . 5

5. Mejora por IA: CARE **5**
5.1. Diferencia con N2V . . . 5
5.2. Generación de pares sintéticos . . . 5
5.3. Entrenamiento . . . 5
5.4. Limitaciones . . . 6

6. Segmentación: Cellpose **6**
6.1. Qué hace . . . 6
6.2. Arquitectura . . . 6
6.3. Cómo separa células que se tocan . . . 6
6.4. Modelo cyto3 . . . 7

7. Segmentación: StarDist **7**
7.1. Principio: distancias radiales . . . 7
7.2. Reconstrucción de polígonos . . . 7
7.3. Arquitectura . . . 7
7.4. Ventajas sobre Cellpose . . . 8
7.5. Limitaciones . . . 8

8. Segmentación: OpenCV (método clásico) **8**
8.1. Proceso paso a paso 8
8.2. Limitaciones 9

9. Detección de fibras **9**

10. Medición dimensional **9**
10.1. Métricas celulares 9
10.2. Métricas de fibras 10
10.3. Estadísticas de resumen 10

11. Comparación de resultados experimentales **10**

12. Consideraciones para despliegue en Raspberry Pi 5 **10**
12.1. Estrategia de despliegue 10
12.2. Presupuesto de recursos 11

13. Reconstrucción FPM (Fourier Ptychographic Microscopy) **11**
13.1. Método multi-ángulo (lensless + OLED) 11
13.2. Otros métodos disponibles 11

14. Resumen de dependencias **12**

1. Visión general del pipeline

El pipeline procesa imágenes de microscopía en 7 etapas secuenciales. Cada imagen pasa por todas las etapas aplicables a su tipo. El objetivo final es obtener mediciones dimensionales calibradas (en micrómetros) de estructuras biológicas.

1. **Carga y clasificación** – Lectura de imágenes y detección automática del tipo de muestra.
2. **Calibración** – Cálculo del factor de escala μm/pixel.
3. **Preprocesamiento** – Mejora de contraste y reducción de ruido clásica.
4. **Mejora por IA** – Denoising avanzado con redes neuronales (opcional).
5. **Segmentación** – Identificación y delimitación de cada célula o fibra individual.
6. **Medición** – Cálculo de métricas dimensionales calibradas.
7. **Exportación** – Generación de reportes, archivos de datos e imágenes anotadas.

Las etapas 4 y 5 ofrecen múltiples métodos (clásicos e IA). Si un modelo de IA falla, el pipeline cae automáticamente al método clásico OpenCV (*graceful degradation*).

2. Calibración

La calibración convierte píxeles a unidades físicas. Sin ella, las mediciones no tienen significado dimensional.

2.1. Método automático (imagen de regla)

El pipeline busca en el lote una imagen que contenga una regla de calibración (microscópica o stage micrometer). El proceso es:

1. Convertir a escala de grises y aplicar detección de bordes Canny.
2. Aplicar la Transformada de Hough para líneas:

$$\rho = x \cos\theta + y \sin\theta \tag{1}$$

   donde ρ es la distancia perpendicular desde el origen a la línea y θ es el ángulo.

3. Filtrar líneas paralelas y calcular la distancia media entre marcas consecutivas en píxeles.
4. Dado que la distancia física entre marcas es conocida (por ejemplo, $10\,\mu$m o $100\,\mu$m), el factor de escala es:

$$S = \frac{d_{\text{física}}[\mu\text{m}]}{d_{\text{píxeles}}[\text{px}]} \quad [\,\mu\text{m/px}] \tag{2}$$

2.2. Método manual

Desde la GUI, el usuario selecciona dos puntos sobre una estructura de tamaño conocido e ingresa la distancia real. El factor se calcula como:

$$S = \frac{d_{\text{real}}[\mu\text{m}]}{\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}\ [\text{px}]} \tag{3}$$

3. Preprocesamiento

Antes de cualquier análisis, cada imagen pasa por dos operaciones clásicas.

3.1. CLAHE (Contrast Limited Adaptive Histogram Equalization)

A diferencia de la ecualización global del histograma, CLAHE divide la imagen en bloques (*tiles*) y ecualiza cada bloque de forma independiente. Esto mejora el contraste local sin saturar zonas brillantes.

- La imagen se divide en bloques de 8×8 píxeles (configurable).
- En cada bloque se calcula el histograma local.
- Se aplica un límite de recorte (*clip limit*, por defecto 2.0) para evitar amplificación excesiva del ruido: los bins del histograma que superan este límite se recortan y se redistribuyen uniformemente.
- Se ecualiza el histograma recortado.
- Se interpola bilinealmente entre bloques adyacentes para evitar artefactos de borde.

3.2. Denoising clásico

Se aplica un filtro bilateral o `fastNlMeansDenoising` de OpenCV. Ambos preservan bordes mejor que un filtro gaussiano estándar:

- **Filtro bilateral**: pondera vecinos por proximidad espacial y por similitud de intensidad. Píxeles con intensidad muy diferente (bordes) reciben peso bajo, preservándose la transición.
- **Non-local means**: para cada píxel, busca parches similares en toda la imagen y promedia los valores. La similitud se mide como distancia euclídea entre parches (ventanas de 7×7).

4. Mejora por IA: Noise2Void (N2V)

4.1. Problema que resuelve

Las imágenes de microscopía tienen ruido intrínseco (ruido de lectura del sensor, ruido térmico, ruido de cuantización). Los filtros clásicos reducen el ruido pero también suavizan bordes y detalles finos. Se necesita un método que elimine ruido preservando las estructuras biológicas.

4.2. Principio fundamental: self-supervised denoising

Noise2Void parte de una observación clave: **el ruido en cada píxel es estadísticamente independiente de sus vecinos, pero la señal no lo es**. Si un píxel contiene información de una pared celular, sus vecinos probablemente también. Pero el ruido en ese píxel no tiene relación con el ruido de los vecinos.

Esto permite entrenar una red neuronal con **una sola imagen ruidosa**, sin necesitar una versión limpia de referencia.

4.3. Arquitectura: U-Net

La red tiene arquitectura U-Net con las siguientes características:

- **Encoder** (camino de contracción): 4 niveles de bloques convolucionales (3×3, ReLU) seguidos de max-pooling 2×2. Cada nivel duplica el número de filtros: $32 \to 64 \to 128 \to 256$.
- **Bottleneck**: bloque convolucional con 512 filtros.
- **Decoder** (camino de expansión): 4 niveles de upsampling 2×2 (transpuesta o bilineal) + concatenación con el mapa del encoder del mismo nivel (*skip connections*) + bloques convolucionales.
- **Salida**: convolución 1×1 con un solo filtro (imagen en escala de grises).
- Total: ∼470,000 parámetros entrenables.

Las *skip connections* son esenciales: permiten que el decoder acceda a los detalles de alta frecuencia del encoder sin tener que reconstruirlos desde la representación comprimida del bottleneck.

4.4. Entrenamiento blind-spot

El truco de N2V es el enmascaramiento *blind-spot*:

Algorithm 1 Entrenamiento Noise2Void

```
Recortar imagen a múltiplo de 32 (requisito de la U-Net)
Extraer 400 parches aleatorios de 64 × 64 píxeles
for cada epoch (10–20 iteraciones) do
    for cada parche P do
        Crear copia P′ del parche
        Seleccionar ∼1 % de píxeles al azar como blind spots
        for cada blind spot en posición (i, j) do
            Guardar valor original: y_ij = P(i, j)
            Reemplazar con valor de un vecino aleatorio: P′(i, j) ← P(i + Δ_i, j + Δ_j)
        end for
        Pasar P′ por la U-Net → predicción P̂
        Calcular pérdida solo en los blind spots:
```

$$\mathcal{L} = \frac{1}{N_{\text{bs}}} \sum_{(i,j)\in\text{blind spots}} \left(\hat{P}(i,j) - y_{ij}\right)^2 \tag{4}$$

```
        Retropropagar y actualizar pesos (Adam, lr = 10^−3)
    end for
end for
```

Por qué funciona: la red nunca ve el valor original del píxel que debe predecir (está enmascarado). Solo puede usar la información de los vecinos. Como el ruido es independiente por píxel, la mejor predicción que la red puede hacer es la señal subyacente (la media condicional dado el contexto). El ruido, al ser aleatorio e incorrelacionado, no puede ser predicho y se cancela.

4.5. Inferencia

Una vez entrenada, la red procesa la imagen completa (con padding a múltiplo de 32) en un solo pase forward. No se aplica enmascaramiento durante la inferencia. La salida es la imagen "limpia" predicha.

Tiempo típico: ∼80 segundos en CPU (10 epochs, imagen 630×1200).

5. Mejora por IA: CARE

5.1. Diferencia con N2V

CARE (*Content-Aware Restoration*) usa la misma arquitectura U-Net que N2V, pero con un enfoque *noise2clean* (supervisado) en lugar de *self-supervised*.

Aspecto	**N2V**	**CARE**
Tipo de entrenamiento	Self-supervised	Supervisado (noise2clean)
Datos de entrada	1 imagen ruidosa	Pares (ruidosa, limpia)
Referencia limpia	No necesita	Generada sintéticamente
Enmascaramiento	Sí (blind-spot)	No
Pérdida	Solo en píxeles enmascarados	En todos los píxeles

Cuadro 1: Comparación N2V vs CARE.

5.2. Generación de pares sintéticos

Como no se dispone de una versión "limpia" de la imagen de microscopía, CARE genera pares de entrenamiento de la siguiente forma:

1. Tomar la imagen original I como referencia "limpia" (tiene ruido real, pero es lo mejor disponible).
2. Generar una versión más ruidosa I_n agregando ruido gaussiano sintético:
$$I_n(x, y) = I(x, y) + \eta(x, y), \quad \eta \sim \mathcal{N}(0, \sigma^2) \tag{5}$$
donde $\sigma = 0{,}3 \times \text{máx}(I)$ (30 % de la intensidad máxima).
3. Extraer 400 parches de 64×64 de ambas versiones, alineados espacialmente.

5.3. Entrenamiento

- **Entrada**: parche ruidoso I_n
- **Target**: parche original I (sin el ruido sintético agregado)
- **Pérdida**: MSE en todos los píxeles del parche:
$$\mathcal{L} = \frac{1}{H \times W} \sum_{x,y} \left(\hat{I}(x, y) - I(x, y) \right)^2 \tag{6}$$
- **Optimizador**: Adam con lr $= 10^{-3}$
- **Epochs**: 10–20

La red aprende a eliminar el componente de ruido gaussiano. Como efecto colateral, también reduce parcialmente el ruido real de la imagen original, ya que el ruido real y el sintético comparten propiedades estadísticas similares.

5.4. Limitaciones

- Asume que el ruido real se comporta de forma similar al ruido gaussiano. Si el ruido es estructurado (patrones fijos del sensor, bandas), CARE no lo eliminará completamente.
- La imagen "limpia" de referencia todavía contiene ruido real. La red aprende a remover solo el ruido agregado, no el preexistente.
- N2V puede ser más efectivo en ciertos casos porque no hace suposiciones sobre la distribución del ruido.

6. Segmentación: Cellpose

6.1. Qué hace

Cellpose toma una imagen de microscopía y produce una máscara de instancias donde cada célula tiene un identificador único (entero). No solo detecta "hay una célula aquí" sino que delimita el contorno exacto de cada una y las separa individualmente, incluso cuando se tocan.

6.2. Arquitectura

Cellpose usa una red residual (basada en ResNet) modificada para predecir tres mapas simultáneamente:

1. **Campo de flujo horizontal** $F_x(i, j)$: para cada píxel, un valor entre -1 y $+1$ que indica la dirección horizontal hacia el centro de la célula a la que pertenece.
2. **Campo de flujo vertical** $F_y(i, j)$: lo mismo en dirección vertical.
3. **Mapa de probabilidad celular** $P(i, j)$: probabilidad de que el píxel pertenezca a una célula (vs fondo).

6.3. Cómo separa células que se tocan

Este es el aporte principal de Cellpose. El proceso es:

1. Para cada célula en la imagen de entrenamiento, se calcula una simulación de difusión térmica (*heat diffusion*) con temperatura fija en el centro de la célula. El gradiente de temperatura resultante apunta desde cualquier punto de la célula hacia su centro.
2. La red aprende a predecir estos gradientes (campos de flujo) directamente desde la imagen.
3. Durante la inferencia, se integran los campos de flujo: cada píxel "sigue" las flechas del flujo durante 200 pasos iterativos. Todos los píxeles que convergen al mismo punto terminal pertenecen a la misma célula.
4. Se filtran píxeles con probabilidad celular $P < 0$ (`cellprob_threshold`).
5. Se filtran flujos con error de convergencia alto (`flow_threshold` $= 0{,}4$).

$$\text{trayectoria}(x, y) = (x, y) + \sum_{t=1}^{200} \left(F_x^{(t)}, F_y^{(t)} \right) \tag{7}$$

Píxeles de células adyacentes que se tocan convergen a centros *diferentes*, lo que los separa naturalmente sin necesidad de post-procesamiento tipo watershed.

6.4. Modelo cyto3

El modelo `cyto3` es la tercera generación del modelo citoplasmático de Cellpose:

- Entrenado con ∼100,000 imágenes de células de múltiples tipos (epiteliales, neuronales, bacterias, levaduras, plantas).
- Tamaño del modelo: ∼25 MB.
- Estima automáticamente el diámetro celular promedio de la imagen.
- Parámetros de inferencia usados en el pipeline:
  - `diameter`: `None` (auto-estimado; resultó en ∼77.7 px en imagen de prueba).
  - `channels`: $[0, 0]$ (escala de grises, sin canal nuclear).
  - `flow_threshold`: 0.4.
  - `cellprob_threshold`: 0.0.
- Resultado en imagen de prueba (630×1200): 162 células detectadas en 38.1 segundos (CPU).

7. Segmentación: StarDist

7.1. Principio: distancias radiales

StarDist aborda la segmentación de forma diferente a Cellpose. En lugar de campos de flujo, predice directamente la forma de cada objeto como un polígono definido por distancias radiales.

Para cada píxel (x, y) que pertenece a un objeto, StarDist predice:

1. n **distancias radiales** $r_k(x, y)$ para $k = 0, 1, \ldots, n-1$ (por defecto $n = 32$). Cada r_k es la distancia desde el píxel hasta el borde del objeto en la dirección angular:

$$\theta_k = \frac{2\pi k}{n} \tag{8}$$

2. **Probabilidad de objeto** $p(x, y)$: si el píxel pertenece a un objeto o al fondo.

7.2. Reconstrucción de polígonos

Con las n distancias radiales desde un píxel central, se reconstruye un polígono de n vértices:

$$V_k = (x + r_k \cos\theta_k,\ y + r_k \sin\theta_k)\,, \quad k = 0, \ldots, n-1 \tag{9}$$

Cada píxel de la imagen que es "objeto" genera un polígono candidato. Luego se aplica *Non-Maximum Suppression* (NMS) para eliminar polígonos redundantes (que representan la misma célula desde píxeles diferentes).

7.3. Arquitectura

- Backbone: U-Net similar a la de N2V/CARE.
- Cabezas de salida:
  - Cabeza de distancias: n canales (un mapa por dirección radial).
  - Cabeza de probabilidad: 1 canal.
- Modelo preentrenado usado: `2D_versatile_fluo` (entrenado en imágenes de fluorescencia variadas).

7.4. Ventajas sobre Cellpose

- **Velocidad**: la inferencia es un solo pase forward por la red + NMS. No requiere los 200 pasos iterativos de integración de flujos de Cellpose. Resultado: ∼4 segundos vs ∼38 segundos en la misma imagen.
- **Polígonos explícitos**: la salida es directamente un contorno poligonal, no una máscara binaria que requiere extracción de contornos posterior.
- **Menor uso de memoria**: no necesita almacenar campos de flujo intermedios.

7.5. Limitaciones

- **Asume objetos convexos**: los polígonos radiales desde un punto central solo pueden representar formas convexas. Células con formas cóncavas o muy irregulares se aproximan pobremente.
- **Resolución angular**: con 32 direcciones, el polígono tiene 32 vértices. Detalles finos del contorno se pierden (un círculo de 32 vértices es una buena aproximación, pero protuberancias pequeñas no se capturan).
- Las células de piel de cebolla son aproximadamente rectangulares (convexas), por lo que StarDist funciona bien en esta aplicación.

8. Segmentación: OpenCV (método clásico)

Este método no usa redes neuronales. Es el fallback cuando los modelos de IA no están disponibles o fallan.

8.1. Proceso paso a paso

1. **Binarización**: umbral adaptativo gaussiano sobre la imagen preprocesada. Divide la imagen en píxeles de primer plano (estructuras) y fondo.
2. **Limpieza morfológica**:
   - *Apertura* (erosión + dilatación): elimina ruido pequeño sin afectar estructuras grandes.
   - *Cierre* (dilatación + erosión): rellena huecos pequeños dentro de las células.
3. **Distance transform**: calcula para cada píxel de primer plano su distancia al borde más cercano. Los centros de las células tienen valores altos; los bordes tienen valores bajos.
4. **Detección de marcadores**: se aplica un umbral sobre el distance transform para encontrar los "picos" (centros de células). Cada pico se etiqueta como un marcador único.
5. **Watershed**: algoritmo de inundación. Cada marcador "crece" como si fuera agua llenando un valle topográfico (definido por el gradiente de la imagen). Cuando dos regiones de agua se encuentran, se levanta una barrera (línea divisoria) que separa las células.
6. **Extracción de contornos**: se extraen los contornos de cada región segmentada con `findContours`.
7. **Filtrado**: se eliminan regiones demasiado pequeñas (ruido) o demasiado grandes (fondo mal segmentado) según umbrales de área configurables.

8.2. Limitaciones

- Sensible al contraste: si los bordes celulares son tenues, la binarización los pierde y las células se fusionan.
- Sobre-segmentación: el watershed tiende a crear líneas divisorias donde no hay bordes reales si los marcadores no están bien ubicados.
- No generaliza: los parámetros deben ajustarse manualmente para cada tipo de muestra.

9. Detección de fibras

Para imágenes de fibra de algodón se usa un pipeline específico:

1. **Detección de bordes** con Canny: detecta transiciones bruscas de intensidad que corresponden a los bordes de las fibras.
2. **Transformada de Hough para líneas**: detecta segmentos de línea recta en la imagen de bordes. Cada fibra aparece como uno o más segmentos de línea.
3. **Esqueletización morfológica**: reduce cada fibra a su eje central de 1 píxel de ancho. Se aplica adelgazamiento iterativo (algoritmo de Zhang-Suen) hasta que cada estructura tiene ancho unitario.
4. **Mediciones**:
   - **Longitud**: número de píxeles del esqueleto $\times$ factor de escala. Para segmentos diagonales se multiplica por $\sqrt{2}$.
   - **Grosor**: distancia promedio desde el esqueleto hasta el borde de la fibra (usando distance transform sobre la máscara binaria de la fibra).
   - **Cruces**: puntos del esqueleto con más de 2 vecinos en la vecindad 3×3 (indican intersecciones entre fibras).

10. Medición dimensional

Todas las mediciones se convierten de píxeles a micrómetros usando el factor de calibración S [μm/px].

10.1. Métricas celulares

Para cada célula segmentada (contorno cerrado):

Métrica	**Fórmula**	**Unidad**	**Interpretación**
Área	$A = N_{\text{px}} \times S^2$	μm^2	Superficie de la célula
Perímetro	$P = L_{\text{contorno}} \times S$	μm	Longitud del borde
Circularidad	$C = \dfrac{4\pi A}{P^2}$	adimensional	1.0 = círculo perfecto

Cuadro 2: Métricas por célula.

Donde N_{px} es el número de píxeles dentro del contorno y L_{contorno} es la longitud del contorno en píxeles (calculada con `arcLength`).

10.2. Métricas de fibras

Métrica	Cálculo	Unidad	Interpretación
Longitud	Píxeles del esqueleto $\times$ S	μm	Largo total de la fibra
Grosor	Distancia media al borde $\times$ $S \times 2$	μm	Diámetro de la fibra
Cruces	Nodos con > 2 vecinos	conteo	Intersecciones entre fibras

Cuadro 3: Métricas por fibra.

10.3. Estadísticas de resumen

Para el lote completo se calculan: media, desviación estándar, mediana, mínimo, máximo y conteo total de cada métrica.

11. Comparación de resultados experimentales

Resultados obtenidos en una imagen de prueba de piel de cebolla (630 $\times$ 1200 píxeles, escala de grises), ejecutando en CPU (Intel, sin GPU):

Método	Células detectadas	Tiempo	Notas
Cellpose cyto3	162	38.1 s	Diámetro auto: 77.7 px
StarDist 2D	$\sim$150	$\sim$4 s	9$\times$ más rápido
N2V + OpenCV	Variable	$\sim$81 s	80.9 s denoising + <1 s segmentación
OpenCV solo	Variable	<1 s	Depende fuertemente del contraste

Cuadro 4: Comparación de métodos en imagen de prueba.

12. Consideraciones para despliegue en Raspberry Pi 5

El objetivo final es ejecutar el pipeline en una Raspberry Pi 5 (4–8 GB RAM, CPU ARM Cortex-A76, sin GPU dedicada).

12.1. Estrategia de despliegue

1. **Exportar a ONNX**: convertir los modelos de PyTorch/TensorFlow a formato ONNX (*Open Neural Network Exchange*) para inferencia optimizada con ONNX Runtime.

2. **Cuantización**: reducir los pesos de float32 a int8 para reducir tamaño del modelo y acelerar inferencia en ARM.

3. **StarDist como modelo principal**: es 9$\times$ más rápido que Cellpose en CPU y produce resultados comparables para células de cebolla (formas convexas).

4. **Graceful degradation**: si ONNX Runtime no está disponible o la inferencia excede el timeout, caer a OpenCV.

12.2. Presupuesto de recursos

Método	Latencia objetivo	RAM estimada	Viable
StarDist ONNX (int8)	<5 s	<500 MB	Sí
Cellpose ONNX	∼15–20 s	∼800 MB	Marginal
N2V entrenamiento	>60 s	∼1 GB	Solo pre-entrenado
OpenCV	<1 s	<100 MB	Sí

Cuadro 5: Estimación de recursos en RPi 5.

13. Reconstrucción FPM (Fourier Ptychographic Microscopy)

Módulo adicional para reconstrucción de imágenes de alta resolución a partir de múltiples capturas de baja resolución tomadas desde diferentes ángulos de iluminación.

13.1. Método multi-ángulo (lensless + OLED)

Diseñado para el setup del CubeSat: sensor de imagen sin lente con pantalla OLED como fuente de iluminación programable.

1. Se capturan N imágenes de baja resolución, cada una iluminada desde un ángulo diferente.
2. Cada ángulo de iluminación desplaza el espectro de frecuencias de la muestra en el espacio de Fourier.
3. El algoritmo itera:
   a) Inicializar la imagen de alta resolución H (upscale por interpolación).
   b) Para cada imagen de baja resolución L_i con ángulo (θ_i, ϕ_i):
      - Calcular la FFT de H.
      - Extraer la región del espectro correspondiente al ángulo i.
      - Reemplazar la amplitud con la de L_i (mantener la fase de H).
      - Actualizar H con la FFT inversa.

   c) Repetir por M iteraciones (típicamente 10–15) hasta convergencia.
4. El resultado es una imagen con resolución efectiva 2× a 4× superior a cada captura individual.

13.2. Otros métodos disponibles

- **Multiframe**: para microscopios con stage motorizado. Usa desplazamientos sub-pixel entre frames para reconstruir resolución superior.
- **Fourier**: ptychography completa con lente de microscopio. Requiere conocer la apertura numérica (NA) del objetivo.

14. Resumen de dependencias

Paquete	**Versión**	**Uso**
opencv-python	$\geq$4.8.0	Preprocesamiento, segmentación clásica, I/O
numpy	$\geq$1.24, <2.0	Operaciones matriciales (incomp. torch con numpy 2)
scikit-image	$\geq$0.21.0	Morfología, esqueletización, métricas
PyYAML	$\geq$6.0	Lectura de configuración
cellpose	$\geq$3.0, <4.0	Segmentación celular (v4 usa CPSAM, inviable en CPU)
tensorflow	$\geq$2.11	Backend para N2V, CARE, StarDist
stardist	$\geq$0.9.0	Segmentación por distancias radiales
csbdeep	$\geq$0.7.2	Dependencia de StarDist
torch	$\geq$2.0	Backend de Cellpose

Cuadro 6: Dependencias principales del pipeline.